

R11

GUIADO EN CLASE

Auditoría de permisos y análisis de texto

Programación para Ciberseguridad · Fase II · AD2026

Cada archivo en Linux tiene permisos para propietario, grupo y otros. Un permiso excesivo (por ejemplo 777) en un archivo sensible es una señal inmediata de alerta. El trío **grep/awk/sed** es la base para automatizar ese tipo de revisión sobre texto plano.

Objetivo: Construir un pipeline de Bash que audite los permisos de una carpeta de prueba y reporte los archivos con configuraciones inseguras.

Instrucciones

1. Crea los archivos de prueba con distintos permisos:

```
mkdir ~/auditoria_permisos && cd ~/auditoria_permisos
touch script_normal.sh config_segura.conf script_riesgoso.sh config_expuesta.conf
chmod 750 script_normal.sh && chmod 644 config_segura.conf
chmod 777 script_riesgoso.sh && chmod 666 config_expuesta.conf
ls -l
```

2. Filtra archivos con escritura abierta para “otros” y extrae su nombre:

```
ls -l | grep -E "^.{8}w"
ls -l | grep -E "^.{8}w" | awk '{print $NF}'
```

3. Genera el reporte con **sed**:

```
ls -l | grep -E "^.{8}w" | awk '{print $NF}' | \
sed 's/~/[RIESGO] Permiso de escritura abierto: /' > reporte_auditoria.txt
cat reporte_auditoria.txt
```

4. Busca además archivos con bit SUID activo:

```
find / -type f -perm -4000 2>/dev/null | head -10
```

Evidencias requeridas

- Screenshot de **ls -l** mostrando los 4 archivos con sus permisos.
- Screenshot del pipeline completo y del archivo **reporte_auditoria.txt** generado.

Preguntas de reflexión

1. ¿Por qué un archivo ejecutable con permiso 777 es más peligroso que un archivo de texto con el mismo permiso?
2. El bit SUID hace que un archivo se ejecute con los permisos de su propietario, no de quien lo corre. ¿Por qué eso es una ruta clásica de escalamiento de privilegios?
3. El pipeline combinó **grep**, **awk** y **sed**. ¿Podrías reemplazar los tres por un solo comando de Python de una línea? ¿Cuándo sería preferible hacerlo así?